ASSIGNMENT 1 — COMPUTER SECURITY COURSE

MINI VAULT

Secure Storage (KV Engine) & Encryption / Signing as a Service (Transit Engine)

I. PRODUCT OVERVIEW

1. Problem Statement

Secrets (DB passwords, API keys...) and sensitive data are often stored in plaintext or encrypted with hardcoded keys embedded in the code. Mini Vault solves two core problems: (1) storing secrets so that data on disk is ALWAYS encrypted and only the rightful owner can read it; (2) allowing other applications to encrypt/decrypt or digitally sign their own data WITHOUT EVER having to hold the encryption or signing key themselves.

2. Two Core Features & Success Criteria

Feature	Required Security Criteria
1. Secure Storage (KV Engine)	Encrypted at rest (data on disk is always ciphertext) + Access-Controlled (only the owner can read/write)
2. Encryption & Signing as a Service (Transit Engine)	The encryption/signing key never leaves the server; ciphertext and signatures carry integrity authentication (AEAD / digital signature); only the owner of a named key can use that key

II. GENERAL RULES

- Each group has 3 students. Clearly state each member's role in the report.
- Recommended language: Python (cryptography, bcrypt/argon2, secrets).
- Code must be clear and commented; avoid copy-pasting libraries of unknown origin.
- Interface: a CLI is sufficient; a REST API (FastAPI/Flask) is encouraged to better reflect a client calling a "service", similar to the real Vault.

Project folder structure (recommended)

```
StudentID1_StudentID2_StudentID3/
├── README.md
├── requirements.txt
├── .env.example
├── main.py
├── src/
│   ├── core/       # Master Passphrase, init/unlock, DEK (section 0.1)
│   ├── auth/        # Register/login, session token (section 0.2)
│   ├── kv/           # Feature 1: Secure Storage
│   ├── transit/       # Feature 2: Encryption & Signing as a Service
│   └── storage/        # Read/write data to disk
├── tests/
├── data/{samples,logs}/
└── docs/report/
```

Naming & packaging rules for submission

- Compress the entire folder into StudentID1_StudentID2_StudentID3.zip.
- Name the report Report_StudentID1_StudentID2_StudentID3.pdf, placed under docs/report/.

- Demo video (recommended) uploaded to Drive/YouTube (Unlisted), with the link included in README.md.

III. REQUIRED FEATURE SPECIFICATION

FEATURE 0 Initialization and Registration / Login

0.1. Vault Initialization & Unlock with Master Passphrase

User Story: *As the person deploying Mini Vault, I set a single Master Passphrase when starting the system for the first time, and must re-enter that exact passphrase every time it restarts to "unlock" the data.*

Functional Flow

1. First run (init): enter a sufficiently strong Master Passphrase. Derive a key from the passphrase using a KDF (Argon2id/PBKDF2, with a randomly generated salt stored separately).
2. Generate a random Data Encryption Key (DEK), encrypt the DEK with the derived key (AES-256-GCM), and write it to disk (encrypted_dek_b64 + salt).
3. On every restart, the default state is "locked" — both Feature 1 (KV) and Feature 2 (Transit) refuse to operate.
4. Re-entering the correct Master Passphrase → re-derive the key → decrypt the DEK → transition to "unlocked".

Data Contract

```
{
  "kdf": "argon2id", "kdf_salt_b64": "<salt>",
  "encrypted_dek_b64": "<encrypted DEK>", "status": "locked"
}
```

Error Cases to Handle

- Wrong Master Passphrase → DEK decryption fails (GCM tag mismatch) → generic error, no detail disclosed.
- Calling any Feature 1 or Feature 2 API while locked → 'VAULT_LOCKED' error.

Acceptance Criteria

- The plaintext DEK must never be written to disk.
- After a restart, the state must always be locked until the correct Master Passphrase is entered.

> **Technical Hint:** *KDF: Argon2id (argon2-cffi) or PBKDF2-HMAC-SHA256. AES-GCM: NIST SP 800-38D.*

0.2. User Identity Authentication

User Story: *As a user, I register and log in so the system knows exactly who I am before letting me touch any secret or key.*

Functional Flow

1. Register: email + passphrase + confirm passphrase. Check passphrase strength and ensure the email doesn't already exist.
2. Hash the passphrase with a dedicated password-hashing algorithm (bcrypt/argon2) — DO NOT hash with plain SHA.
3. Login: email + passphrase matching the stored hash → issue a session token with an expiry (e.g., 30 minutes), used for every request to Feature 1 and Feature 2.

4. 5 consecutive failed passphrase attempts → temporarily lock the account for 5 minutes (mandatory).

Data Contract

```
{ "email": "alice@example.com", "password_hash": "<bcrypt/argon2>", ... }
```

Error Cases to Handle

- Account does not exist at login → error message.
- Session token expired → require re-login.

Acceptance Criteria

- Every Feature 1 and Feature 2 API call must require a valid session token; no endpoint may skip this check.
- Required test: deliberately enter the wrong passphrase 5 times in a row → the account must be locked for exactly 5 minutes, and login must fail even with the correct passphrase while locked.

> **Technical Hint:** *Library: bcrypt or argon2-cffi.*

FEATURE 1 Secure Storage — KV Engine

1.1. Encrypted-at-Rest Storage

Security Guarantee: Anyone who obtains the raw data file (leaked backup, lost USB drive...) without the DEK CANNOT read the secret's content, even if the vault is unlocked on another machine.

User Story: *As a user, I want to store a secret (e.g., a DB password) under a name (path) and retrieve the exact same content later, without worrying about exposure if the storage file falls into someone else's hands.*

Functional Flow

1. Client calls write(path, data), where data is any JSON object, along with a session token.
2. The system encrypts the entire data payload using AEAD (AES-256-GCM) with the current DEK, generating a fresh random nonce for every write (never reusing a nonce with the same key).
3. Write the ciphertext + nonce + tag to disk; an existing path (if any) is overwritten directly, with NO version history kept (kept simple — see section IV for extra credit if you want versioning).
4. Client calls read(path) → the system decrypts using the DEK and verifies the tag before returning data; if the tag is invalid → refuse outright, never returning data that "might be right, might be wrong".
5. Client calls delete(path) → permanently deletes the record.

Input / Output

API	Input	Output
write	path, data (JSON), token	created_at / updated_at
read	path, token	decrypted data (only if the tag is valid)
delete	path, token	deletion confirmation

Data Contract

```
{ "path": "secret/alice@example.com/db", "nonce_b64": "...", "ciphertext_b64": "...", "tag_b64":
"..." }
```

Error Cases to Handle

- Write/read while the vault is locked → 'VAULT_LOCKED'.
- Authentication tag mismatch on read (data on disk was manually tampered with) → refuse to decrypt and do not return the secret.
- Reading a path that doesn't exist → 'NOT_FOUND', no garbage data returned.

Acceptance Criteria

- write then read on the same path must return the original data exactly (round-trip test).
- Opening the data file directly in a text editor: no plaintext fragment of any secret should be visible.
- Test: manually altering 1 byte in the ciphertext or tag on disk → read must refuse, never returning corrupted data.

> **Technical Hint:** *AES-256-GCM: NIST SP 800-38D. Do not use CBC without integrity authentication.*

1.2. Ownership-based Access Control

Security Guarantee: A valid session token belonging to user A cannot read/write/delete a secret in user B's namespace, even by guessing the correct path.

User Story: *As a user, I am guaranteed that only I (the owner) can read/write/delete my own secrets, even if someone else knows the exact path.*

Functional Flow

1. Every secret is stored under a path with a fixed owner prefix: secret/<email>/....
2. Every write/read/delete request (section 1.1) checks whether the email in the session token matches the email in the path prefix.
3. If they don't match → refuse immediately BEFORE touching any encryption/decryption operation, returning a generic error that doesn't distinguish between "path doesn't exist" and "no permission" (to avoid leaking which paths are in use).
4. Log every denied access attempt along with the requester's email and the denied path.

Error Cases to Handle

- Valid token but the path doesn't belong to the caller's own namespace → 'PERMISSION_DENIED' (without disclosing whether that path exists).
- Token expired or invalid → 'UNAUTHENTICATED', rejected before permission is even evaluated.

Acceptance Criteria

- Required test: user A, using their own valid token, attempts to read secret/<B's email>/... → must be denied 100% of the time.
- Test: a request with a missing/completely invalid token must never reach the path-check step.

> **Technical Hint:** *This is a minimal (ownership-based) access control model.*

FEATURE 2 Encryption & Signing as a Service — Transit Engine

2.1. Named Key Management

Security Guarantee: The key used to encrypt a client's data is NEVER returned through any API, under any form (even to its own owner) — it can only be used indirectly via encrypt/decrypt.

User Story: *As a user, I want to create a named key dedicated to encrypting my own data, without having to generate and safeguard the AES key myself.*

Functional Flow

1. Client calls create_key(key_name) with a token → the system generates a random AES-256 key, bound to key_name and the owner (email from the token). This key is stored with key_usage = "ENCRYPT_DECRYPT" — mirroring AWS KMS's KeyUsage attribute — to distinguish it from the signing keys created in section 2.4.
2. This AES key is encrypted with the DEK (section 0.1) before being written to disk — named keys must also be encrypted at rest, exactly like the requirement in Feature 1.
3. The client can call list_keys() to see the names and key_usage of keys they've created (never the real key material), and revoke_key(key_name) to permanently delete a named key.

Data Contract

```
{ "key_name": "my-key", "owner_email": "alice@example.com", "key_usage": "ENCRYPT_DECRYPT",
  "encrypted_key_material_b64": "<AES key encrypted with the DEK>" }
```

Error Cases to Handle

- Creating a key_name that already exists (owned by the same user) → prompt for overwrite confirmation, or reject and ask for a different name (group's choice, to be documented in the report).
- Creating/deleting a key while the vault is locked → 'VAULT_LOCKED'.

Acceptance Criteria

- No API (including list_keys) ever returns the real AES key in plaintext or base64 form.

> **Technical Hint:** *Refer to the real Transit named-key model: https://developer.hashicorp.com/vault/docs/secrets/transit · KeyUsage concept: https://docs.aws.amazon.com/kms/latest/APIReference/API_CreateKey.html*

2.2. Encryption / Decryption as a Service (Encrypt & Decrypt API)

User Story: *As a user, I send raw data to Mini Vault to be encrypted with my named key, and later send back the ciphertext to be decrypted at any time, without needing to know the real key.*

Functional Flow

1. Client calls encrypt(key_name, plaintext_b64, token) → the system verifies ownership of key_name (see 2.3), temporarily decrypts the AES key using the in-memory DEK, generates a random nonce, and encrypts the plaintext (AES-256-GCM).
2. Returns a self-describing ciphertext containing key_name + nonce + ciphertext + tag, so the client never needs to remember which key was used.
3. Client calls decrypt(ciphertext, token) → the system reads the key_name embedded in the ciphertext, checks permission, decrypts the corresponding AES key using the DEK, decrypts and verifies the tag, and returns the plaintext.

Input / Output

API	Input	Output
encrypt	key_name, plaintext (base64), token	ciphertext of the form 'vault:<key_name>:<base64(nonce+ct+tag)>'
decrypt	ciphertext, token	plaintext (base64)

Error Cases to Handle

- Malformed or truncated ciphertext → refuse to decrypt.
- GCM tag mismatch (ciphertext has been tampered with) → refuse, with a clear reason.
- key_name doesn't exist or has been revoked → refuse both encrypt and decrypt.
- key_name exists but its key_usage is "SIGN_VERIFY" (created in section 2.4), not "ENCRYPT_DECRYPT" → reject with a clear error, mirroring AWS KMS's InvalidKeyUsageException.

Acceptance Criteria

- encrypt followed by decrypt must return the exact original plaintext (round-trip test) across multiple data types (text, JSON, binary base64).
- Altering any single byte in the ciphertext → decrypt must fail clearly, 100% of the time.
- No request (encrypt/decrypt/list_keys) ever returns the real AES key.

> **Technical Hint:** *Refer to the real Transit Secrets Engine: https://developer.hashicorp.com/vault/docs/secrets/transit*

2.3. Named-Key Access Control

Security Guarantee: A user cannot use another user's named key to encrypt/decrypt, even if they know the exact key name.

User Story: *As a user, I am guaranteed that only I can use my own named key to encrypt/decrypt, even if someone else knows the key's name.*

Functional Flow

1. Every named key is stored together with an owner_email (section 2.1).
2. Every encrypt/decrypt request (section 2.2) checks whether the email in the token matches the owner_email of the key_name being called.
3. If they don't match → refuse, WITHOUT performing any encryption/decryption operation, returning a generic error (without disclosing whether that key_name exists).
4. Log every denied access attempt along with the requester's email and the denied key_name.

Error Cases to Handle

- Valid token but not the owner of key_name → 'PERMISSION_DENIED'.

Acceptance Criteria

- Required test: user A attempts to encrypt/decrypt using a key_name owned by user B → must be denied 100% of the time.

2.4. Sign & Verify as a Service

Security Guarantee: The private signing key never leaves the server — exactly like the encryption key in 2.1. Verification must reject any message that was altered after signing, and any signature produced with a different key, without ever exposing the private key to do so.

User Story: *As a user, I want to digitally sign a message with my own signing key, and let anyone verify that the signature is authentic and the message hasn't been tampered with, without ever handling the private key myself — modeled after AWS KMS's Sign / Verify APIs.*

Functional Flow

1. Client calls create_signing_key(key_name, signing_algorithm) with a token → the system generates an asymmetric key pair (e.g., RSA-2048 for RSASSA_PKCS1_V1_5_SHA_256, or ED25519). The private key is encrypted with the DEK before being stored (same pattern as 2.1); the public key is stored so the server can verify later — it is still never exposed to a client that isn't authorized to see it.
2. Client calls sign(key_name, message_b64, message_type, token), where message_type is either RAW (the system hashes the message with SHA-256 first) or DIGEST (the client already sends a precomputed hash) — mirroring AWS KMS's Sign API parameters. The system checks ownership of key_name, signs using the private key in memory (decrypted only temporarily via the DEK), and returns the signature.
3. Client calls verify(key_name, message_b64, message_type, signature_b64, token) → the system recomputes/uses the digest the same way as in sign(), checks the signature against the key's public component, and returns a structured result — {key_name, signature_valid, signing_algorithm} — mirroring

the response shape of AWS KMS's Verify API (KeyId, SignatureValid, SigningAlgorithm), instead of silently assuming a signature is correct.

4. Apply the same ownership-based access control as 2.1/2.3: only the key's owner may call sign(); the mandatory scope also requires the caller to be the owner to call verify() (opening verify() to other authenticated users is a valid extension — see section IV).

Input / Output

API	Input	Output
create_signing_key	key_name, signing_algorithm, token	confirmation
sign	key_name, message (base64), message_type (RAW|DIGEST), token	signature (base64), key_name, signing_algorithm
verify	key_name, message (base64), message_type, signature (base64), token	key_name, signature_valid (boolean), signing_algorithm

Data Contract

```
{
  "key_name": "my-signing-key", "owner_email": "alice@example.com", "key_usage": "SIGN_VERIFY",
  "signing_algorithm": "ED25519",
  "encrypted_private_key_b64": "<private key encrypted with the DEK>",
  "public_key_b64": "<public key, used internally by the server to verify>"
}
```

Error Cases to Handle

- message_type = DIGEST but the digest length doesn't match the expected hash output size → reject the request.
- verify() called with a signing_algorithm that doesn't match the one the key was created with → reject.
- key_name doesn't exist or has been revoked → refuse both sign and verify.
- key_name exists but its key_usage is "ENCRYPT_DECRYPT" (created in section 2.1), not "SIGN_VERIFY" → reject with a clear error, mirroring AWS KMS's InvalidKeyUsageException.
- Malformed or wrong-length signature passed to verify() → return signature_valid: false (or a clear rejection), never throw an unhandled exception.

Acceptance Criteria

- sign() followed by verify() on the unmodified message must return signature_valid: true, 100% of the time.
- Altering a single byte of the message before calling verify() → signature_valid must be false, 100% of the time.
- Using a signature produced by one named key to verify() against a different named key → signature_valid must be false.
- No API ever returns the raw private signing key.

> **Technical Hint:** *Modeled after AWS KMS: Sign — https://docs.aws.amazon.com/kms/latest/APIReference/API_Sign.html and Verify — https://docs.aws.amazon.com/kms/latest/APIReference/API_Verify.html. Python libraries: cryptography (rsa / ed25519 modules).*

IV. ADVANCED FEATURES (OPTIONAL — EXTRA CREDIT)

Should only be attempted after all 8 required sub-features in section III are running stably. Total extra credit from this section may not exceed 1.0 point out of 10.

Advanced Feature	Suggested Extra Credit
Sharing named keys/secrets across multiple users via a full Policy/ACL system (instead of simple ownership-based control)	+0.4
MFA (OTP/TOTP) for the login step in section 0.2	+0.2
Shamir's Secret Sharing for section 0.1 (replacing a single Master Passphrase with N key shares, requiring K shares)	+0.5
Key rotation for Transit (versioned named keys, still able to decrypt old ciphertext)	+0.4
KV versioning (keeping a history of overwrites)	+0.3
Tamper-evident audit log (hash-chained, detects log tampering)	+0.3
Opening verify() in 2.4 to any authenticated user, not just the key owner (with an explicit share/grant model, similar to AWS KMS key policies)	+0.3

V. SUGGESTED TECHNOLOGY (reference)

Task	Suggested Python Library
Storing KV / named keys / users	sqlite3 (standard library) or JSON
Password hashing	bcrypt, argon2-cffi
Key derivation from the Master Passphrase (KDF)	argon2-cffi or hashlib.pbkdf2_hmac
AES-256-GCM encryption	cryptography (AESGCM) or pycryptodome
Asymmetric signing (RSA / ED25519)	cryptography (rsa, ed25519 modules)
Cryptographically secure random generation (nonce, key, salt)	secrets, os.urandom
REST API (optional)	FastAPI or Flask
Unit testing / CI (optional)	pytest, GitHub Actions

VI. SUBMISSION MUST INCLUDE

- Full source code, organized into modules as described in section II, with a README.md.
- Report: team name, student IDs, task assignment; architecture diagram; technical explanation for sections 0.1, 0.2, 1.1, 1.2, 2.1, 2.2, 2.3, 2.4; screenshots of the demo; optional features completed (if any).
- Demo video (recommended, 3–5 minutes): unlock, write/read a secret, attempting to access another user's secret (denied), creating a named key, encrypt/decrypt, attempting to use another user's key (denied), sign a message, verify it (valid), then verify a tampered message (invalid).
- Test data files: an encrypted KV data file, sample ciphertext from Transit.

VII. GRADING RUBRIC

No.	Category	Detailed Content & Grading Criteria	Points
1	0.1 — Init & Unlock	Correct KDF, defaults to locked after restart, no plaintext DEK leaked	1.0
2	0.2 — User Authentication	Correct password hashing, session token, temporary lockout after 5 failed attempts	1.0
3	1.1 — KV Encrypted-at-Rest	Correct AEAD, detects tampered on-disk data, no plaintext leaked	1.25
4	1.2 — KV Access Control	Blocks 100% of unauthorized cross-user access	1.0
5	2.1 — Transit Key Management	Named key never returned in plaintext via API	1.0
6	2.2 — Transit Encrypt/Decrypt	Correct round-trip, detects tampered ciphertext	1.25
7	2.3 — Transit Access Control	Blocks 100% of attempts to use another user's key	1.0
8	2.4 — Sign & Verify	Correct round-trip, rejects tampered messages and cross-key signatures 100% of the time	1.0
9	Report, README, task assignment	Clear, complete, well-illustrated, with run instructions	0.75
10	Product Demo	Clear demo, including denied-access cases and sign/verify cases	0.75